

30 requêtes SQL piégées corrigées, ligne par ligne



Le réflexe à avoir avant qu'un test technique Data Analyst ne te fasse écrire l'une d'elles.

Un test SQL en entretien ne sanctionne pas l'ignorance : il sanctionne le réflexe d'instinct qui casse un chiffre sans lever d'erreur. Ce guide prend 30 requêtes que tout junior écrit naturellement, montre où chacune casse, la corrige ligne par ligne, et te donne le signal à retenir pour le jour J. Dialecte : PostgreSQL.

Observation Datafuji : sur les tests techniques que j'ai vus passer, la moitié des candidats n'échouent pas sur la syntaxe. Ils échouent sur une requête qui s'exécute sans erreur et renvoie un chiffre faux. Schéma utilisé dans tout le guide : **clients, commandes, lignes_commande, produits, employes**.

Filter et compter juste.

Ici, le piège n'est presque jamais une erreur de syntaxe : c'est un chiffre qui a l'air juste et qui ne l'est pas.

```
SELECT COUNT(DISTINCT id) AS nb_clients  
FROM commandes;
```

1. DISTINCT sur la mauvaise colonne PIÈGE CLASSIQUE

Pourquoi ça casse : **id** est déjà la clé primaire de **commandes**, donc déjà unique. DISTINCT ne retire aucun doublon : le résultat est le nombre total de commandes, pas le nombre de clients différents.

```
SELECT COUNT(DISTINCT client_id) AS nb_clients  
FROM commandes;
```

Signal à retenir : avant de poser DISTINCT, demande-toi sur quelle colonne portent les doublons que tu veux enlever, jamais sur la requête entière.

```
SELECT COUNT(date_livraison) AS nb_commandes  
FROM commandes;
```

2. COUNT(colonne) vs COUNT(*) PIÈGE CLASSIQUE

Pourquoi ça casse : COUNT(colonne) ignore les lignes où **date_livraison** est NULL, c'est à dire les commandes pas encore livrées. Le total affiché est sous évalué, sans aucune erreur visible.

```
SELECT COUNT(*) AS nb_commandes,  
       COUNT(date_livraison) AS nb_livrees  
FROM commandes;
```

Signal à retenir : COUNT(*) compte les lignes, COUNT(colonne) compte les valeurs non NULL de cette colonne. Choisis selon ce que tu veux vraiment mesurer.

```
SELECT client_id, SUM(montant) AS total
FROM commandes
WHERE SUM(montant) > 500
GROUP BY client_id;
```

3. WHERE sur un agrégat PIÈGE CLASSIQUE

Pourquoi ça casse : le WHERE s'exécute avant le GROUP BY, l'agrégat SUM n'existe pas encore à ce moment. PostgreSQL refuse la requête : **aggregate functions are not allowed in WHERE**.

```
SELECT client_id, SUM(montant) AS total
FROM commandes
GROUP BY client_id
HAVING SUM(montant) > 500;
```

Signal à retenir : un filtre sur une valeur agrégée va toujours dans HAVING, jamais dans WHERE.

```
SELECT nom, email
FROM clients
WHERE email LIKE 'gmail.com';
```

4. LIKE mal wildcardé PIÈGE CLASSIQUE

Pourquoi ça casse : sans %, LIKE exige une correspondance exacte de toute la chaîne. La requête ne renvoie jamais aucun résultat, sans lever la moindre erreur.

```
SELECT nom, email
FROM clients
WHERE email LIKE '%gmail.com';
```

Signal à retenir : LIKE sans wildcard est un signe égal déguisé. Place le % du bon côté selon ce que tu cherches : début, fin ou milieu de chaîne.

```
SELECT nom
FROM clients
WHERE code_postal = 75001;
```

5. Comparaison implicite chaîne/nombre PIÈGE CLASSIQUE

Pourquoi ça casse : **code_postal** est stocké en VARCHAR, pense aux codes qui commencent par 0. PostgreSQL refuse de comparer texte et entier sans cast explicite : erreur **operator does not exist**. Portabilité : MySQL, lui, caste silencieusement et laisse passer une comparaison qui peut être fausse, sans erreur visible.

```
SELECT nom
FROM clients
WHERE code_postal = '75001';
```

Signal à retenir : une erreur de type au test est presque un cadeau, elle t'alerte tout de suite. Vérifie le type réel de la colonne avant de comparer.

Jointures qui trompent.

Ici, le recruteur vérifie une chose simple : est-ce que ta jointure garde, perd ou multiplie les bonnes lignes ?

À retenir avant le bloc JOIN

Le mot LEFT JOIN rassure, mais un filtre mal placé ou une clé mal choisie peuvent annuler exactement ce que tu voulais garder.

Avant de joindre

La clé de jointure est-elle unique côté droit ?

Après avoir joint

Compare le nombre de lignes avant et après.

```
SELECT c.nom, o.date_commande
FROM clients c
INNER JOIN commandes o ON o.client_id = c.id;
```

6. INNER JOIN qui efface des lignes PIÈGE CLASSIQUE

Pourquoi ça casse : INNER JOIN ne garde que les clients qui ont au moins une commande correspondante. Les clients sans commande disparaissent silencieusement du résultat.

```
SELECT c.nom, o.date_commande
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id;
```

Signal à retenir : dès que l'absence de correspondance est une information utile, pense LEFT JOIN, pas INNER JOIN.

```
SELECT c.nom, o.id AS commande_id
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id
WHERE o.statut = 'payee';
```

7. LEFT JOIN annulé par un WHERE PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : le WHERE s'applique après la jointure et exige une valeur sur **o.statut**. Les clients sans commande, où cette colonne est NULL, sont éliminés : ton LEFT JOIN se comporte comme un INNER JOIN, sans le dire.

```
SELECT c.nom, o.id AS commande_id
FROM clients c
LEFT JOIN commandes o
  ON o.client_id = c.id AND o.statut = 'payee';
```

Signal à retenir : un filtre sur la table de droite d'un LEFT JOIN va dans le ON s'il doit rester optionnel, jamais dans le WHERE.

```
SELECT o.id, o.montant, l.quantite
FROM commandes o
JOIN lignes_commande l ON l.produit_id = o.id;
```

8. Jointure sur la mauvaise clé PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : **produit_id** n'a aucun rapport avec l'identifiant de commande. La jointure associe des lignes selon une coïncidence de valeurs, et un même id de commande peut matcher plusieurs produits, dupliquant les montants dans les agrégats suivants.

```
SELECT o.id, o.montant, l.quantite
FROM commandes o
JOIN lignes_commande l ON l.commande_id = o.id;
```

Signal à retenir : avant d'écrire un JOIN, vérifie que la colonne de gauche et celle de droite désignent bien la même entité, pas juste des noms qui se ressemblent.

```
SELECT id, nom
FROM clients c
JOIN commandes o ON o.client_id = c.id
JOIN lignes_commande l ON l.commande_id = o.id;
```

9. Colonne ambiguë multi-JOIN PIÈGE CLASSIQUE

Pourquoi ça casse : `id` existe dans `clients`, `commandes` et `lignes_commande`. PostgreSQL renvoie **column reference id is ambiguous**, car il ne sait pas laquelle choisir.

```
SELECT c.id AS client_id, c.nom
FROM clients c
JOIN commandes o ON o.client_id = c.id
JOIN lignes_commande l ON l.commande_id = o.id;
```

Signal à retenir : dès la deuxième table jointe, préfixe systématiquement chaque colonne par son alias, même quand le nom semble sans ambiguïté aujourd'hui.

```
SELECT c.id, COUNT(*) AS nb_commandes
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id
GROUP BY c.id;
```

10. COUNT après LEFT JOIN PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : `COUNT(*)` compte les lignes du résultat de la jointure, y compris celles où la partie commande est entièrement NULL. Chaque client sans commande affiche 1 au lieu de 0.

```
SELECT c.id, COUNT(o.id) AS nb_commandes
FROM clients c
LEFT JOIN commandes o ON o.client_id = c.id
GROUP BY c.id;
```

Signal à retenir : après un LEFT JOIN, compte une colonne de la table de droite, pas `COUNT(*)` : `COUNT(colonne)` ignore les NULL, `COUNT(*)` non.

```
SELECT a.nom, b.nom, a.departement
FROM employes a
JOIN employes b
  ON a.departement = b.departement AND a.id <> b.id;
```

11. Self JOIN mal conditionné PIÈGE CLASSIQUE

Pourquoi ça casse : chaque paire apparaît deux fois, (a, b) et (b, a). Le volume de résultat double sans qu'aucune erreur ne le signale.

```
SELECT a.nom, b.nom, a.departement
FROM employes a
JOIN employes b
  ON a.departement = b.departement AND a.id < b.id;
```

Signal à retenir : dans un self JOIN, remplace <> par < sur la clé pour ne garder qu'un seul sens de chaque paire.

Agrégations et GROUP BY.

Ici, une requête qui s'exécute sans erreur n'est pas forcément une requête juste.

```
SELECT client_id, date_commande, SUM(montant) AS total
FROM commandes
GROUP BY client_id;
```

12. GROUP BY incomplet PIÈGE CLASSIQUE

Pourquoi ça casse : `date_commande` n'est ni agrégée ni dans le GROUP BY. PostgreSQL refuse : **column must appear in the GROUP BY clause**. Portabilité : MySQL sans ONLY_FULL_GROUP_BY laisserait passer, en choisissant une date arbitraire par groupe, un chiffre faux mais juste en apparence.

```
SELECT client_id, SUM(montant) AS total
FROM commandes
GROUP BY client_id;
```

Signal à retenir : chaque colonne du SELECT est soit dans le GROUP BY, soit dans une fonction d'agrégation. Pas de troisième option en PostgreSQL.

```
SELECT client_id, COUNT(*) AS nb
FROM commandes
GROUP BY client_id
WHERE statut = 'payee';
```

13. WHERE confondu avec HAVING PIÈGE CLASSIQUE

Pourquoi ça casse : erreur de syntaxe pure. L'ordre logique impose WHERE avant GROUP BY : PostgreSQL refuse la requête telle quelle.

```
SELECT client_id, COUNT(*) AS nb
FROM commandes
WHERE statut = 'payee'
GROUP BY client_id;
```

Signal à retenir : l'ordre d'écriture suit l'ordre logique, SELECT puis FROM, WHERE, GROUP BY, HAVING, ORDER BY. Un filtre sur une colonne brute vient toujours avant le GROUP BY.

```
SELECT AVG(note_satisfaction) AS moyenne
FROM commandes;
```

14. AVG qui ignore les NULL PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : AVG ignore les lignes où **note_satisfaction** est NULL, un client qui n'a pas répondu. La moyenne porte seulement sur les répondants, pas sur toutes les commandes, ce qui gonfle le résultat si les non répondants sont plutôt insatisfaits.

```
SELECT SUM(COALESCE(note_satisfaction, 0)) / COUNT(*) AS moyenne_sur_toutes,  
       AVG(note_satisfaction) AS moyenne_sur_repondants  
FROM commandes;
```

Signal à retenir : avant d'afficher une moyenne, précisez si elle porte sur toutes les lignes ou seulement sur celles avec une valeur. Ce sont deux chiffres différents.

```
SELECT SUM(o.montant) AS total_commandes  
FROM commandes o  
JOIN lignes_commande l ON l.commande_id = o.id;
```

15. SUM après jointure dupliquée PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : après la jointure, **o.montant** est répété sur chaque ligne de **lignes_commande**. Une commande à 3 lignes affiche 3 fois le même montant : le SUM le compte 3 fois, un double comptage silencieux.

```
SELECT SUM(montant) AS total_commandes  
FROM commandes;
```

Signal à retenir : après une jointure qui multiplie les lignes, ne somme jamais une colonne de la table côté « un » sans regrouper dessus, ou calcule-la dans une requête séparée sur la table d'origine.

```
SELECT EXTRACT(MONTH FROM date_commande) AS mois,  
       SUM(montant) AS total  
FROM commandes  
GROUP BY mois;
```

16. Mauvaise granularité de GROUP BY PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : la requête est syntaxiquement valide et s'exécute sans erreur, mais EXTRACT(MONTH ...) fusionne janvier 2024 et janvier 2025 dans le même groupe « 1 ». Le total mensuel mélange deux années, sans aucune alerte.

```
SELECT DATE_TRUNC('month', date_commande) AS mois,  
       SUM(montant) AS total  
FROM commandes  
GROUP BY mois  
ORDER BY mois;
```

Signal à retenir : une requête qui tourne sans erreur n'est pas forcément juste. Vérifie que le grain du GROUP BY correspond vraiment à la question métier posée.

NULL et logique à trois valeurs.

Ici, NULL n'est ni vrai ni faux. C'est ce troisième état qui piège la moitié des candidats.

```
SELECT nom
FROM clients
WHERE date_derniere_commande = NULL;
```

17. Colonne = NULL PIÈGE CLASSIQUE

Pourquoi ça casse : NULL représente une valeur inconnue. Une comparaison avec = NULL ne renvoie jamais VRAI, même pour une ligne réellement NULL. La requête tourne sans erreur et renvoie toujours zéro ligne.

```
SELECT nom
FROM clients
WHERE date_derniere_commande IS NULL;
```

Signal à retenir : pour tester une valeur manquante, utilise toujours IS NULL ou IS NOT NULL. Le signe = ne teste jamais l'absence de valeur.

```
SELECT nom
FROM clients c
WHERE id NOT IN (
    SELECT client_id FROM commandes WHERE statut = 'annulee'
);
```

18. NOT IN avec NULL dans la sous-liste PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : si la sous-requête renvoie ne serait-ce qu'une seule valeur NULL, comme une commande orpheline sans client, NOT IN devient indécidable pour chaque ligne comparée à ce NULL. PostgreSQL renvoie zéro résultat pour toute la requête, silencieusement.

```
SELECT nom
FROM clients c
WHERE NOT EXISTS (
    SELECT 1 FROM commandes o
    WHERE o.client_id = c.id AND o.statut = 'annulee'
);
```

Signal à retenir : dès qu'une sous-liste peut contenir un NULL, remplace NOT IN par NOT EXISTS, qui reste fiable quel que soit le contenu de la sous-requête.

```
SELECT id, COALESCE(montant, 0) AS montant_affiche
FROM commandes
WHERE statut = 'payee'
ORDER BY montant DESC;
```

19. COALESCE mal placé PIÈGE CLASSIQUE

Pourquoi ça casse : le ORDER BY porte sur la colonne brute **montant**, pas sur le résultat de COALESCE. Les lignes avec montant NULL, déjà remplacées par 0 à l'affichage, se trient n'importe où selon la convention NULLS FIRST ou LAST du moteur.

```
SELECT id, COALESCE(montant, 0) AS montant_affiche
FROM commandes
WHERE statut = 'payee'
ORDER BY COALESCE(montant, 0) DESC;
```

Signal à retenir : si tu remplaces une valeur avec COALESCE quelque part dans la requête, applique le même COALESCE partout où cette colonne est encore utilisée, filtre ou tri, pas seulement à l'affichage.

```
SELECT nom
FROM clients
WHERE ville = 'Lyon' OR ville <> 'Lyon';
```

20. OR avec NULL en condition composée PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : pour un client dont **ville** est NULL, les deux comparaisons valent UNKNOWN, et UNKNOWN OR UNKNOWN reste UNKNOWN, pas VRAI. Ces clients disparaissent alors que la requête semblait couvrir « tous les cas ».

```
SELECT nom
FROM clients
WHERE ville = 'Lyon' OR ville <> 'Lyon' OR ville IS NULL;
```

Signal à retenir : une condition qui semble couvrir « tout » en logique classique ne couvre jamais le cas NULL. Ajoute toujours IS NULL explicitement si tu veux vraiment tout inclure.

IS NULL contre = NULL

Le piège le plus fréquent de ce palier, résumé en une comparaison.

= NULL

Ne renvoie jamais VRAI, même sur une ligne NULL : la comparaison vaut UNKNOWN.

Résultat : zéro ligne, sans erreur.

IS NULL

Teste correctement l'absence de valeur, VRAI ou FAUX sans ambiguïté.

Résultat : les bonnes lignes.

Sous-requêtes et CTE.

Ici, la requête tourne presque toujours. La question est de savoir si elle répond à la bonne échelle.

```
SELECT c.nom,
       (SELECT MAX(date_commande) FROM commandes) AS derniere_commande
FROM clients c;
```

21. Sous-requête corrélée mal référencée PIÈGE CLASSIQUE

Pourquoi ça casse : la sous-requête ne référence jamais **c.id**. Elle n'est pas corrélée : PostgreSQL calcule une seule fois le max sur toutes les commandes et l'affiche identique pour chaque client, au lieu d'une date par client.

```
SELECT c.nom,
       (SELECT MAX(o.date_commande) FROM commandes o
        WHERE o.client_id = c.id) AS derniere_commande
FROM clients c;
```

Signal à retenir : si une sous-requête doit varier ligne par ligne, vérifie qu'elle référence bien une colonne de la requête externe dans son WHERE.


```
SELECT nom
FROM produits
WHERE id NOT IN (SELECT produit_id FROM lignes_commande);
```

22. EXISTS vs IN avec NULL PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : même piège que NOT IN sur des commandes, transposé aux produits. Si **produit_id** peut être NULL sur des lignes de service, un seul NULL suffit à faire renvoyer zéro ligne à toute la requête.

```
SELECT nom
FROM produits p
WHERE NOT EXISTS (
    SELECT 1 FROM lignes_commande l WHERE l.produit_id = p.id
);
```

Signal à retenir : NOT IN et les colonnes nullable ne font jamais bon ménage. NOT EXISTS est le réflexe par défaut pour « n'existe pas dans ».

```
WITH totaux AS (
    SELECT client_id, SUM(montant) AS total
    FROM commandes
    GROUP BY client_id
)
SELECT client_id, total
FROM totaux
WHERE total > (SELECT AVG(montant) FROM commandes);
```

23. CTE mal référencée PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : la moyenne est calculée sur **commandes** ligne par ligne, un montant par commande, pas sur les totaux par client de la CTE. On compare un total cumulé à une moyenne de commandes individuelles, deux échelles différentes, sans que la requête ne s'en plaigne.

```
WITH totaux AS (
    SELECT client_id, SUM(montant) AS total
    FROM commandes
    GROUP BY client_id
)
SELECT client_id, total
FROM totaux
WHERE total > (SELECT AVG(total) FROM totaux);
```

Signal à retenir : quand tu compares à une moyenne, vérifie que le AVG porte sur la même unité que ce que tu compares, pas sur la table brute par réflexe.

```
SELECT c.nom,
    (SELECT p.nom FROM lignes_commande l
     JOIN produits p ON p.id = l.produit_id
     JOIN commandes o ON o.id = l.commande_id
     WHERE o.client_id = c.id) AS dernier_produit
FROM clients c;
```

24. Sous-requête scalaire multi-lignes PIÈGE CLASSIQUE

Pourquoi ça casse : dès qu'un client a commandé plus d'un produit, la sous-requête scalaire renvoie plusieurs lignes. PostgreSQL lève **more than one row returned by a subquery** et toute la requête échoue.

```
SELECT c.nom,
       (SELECT p.nom FROM lignes_commande l
        JOIN produits p ON p.id = l.produit_id
        JOIN commandes o ON o.id = l.commande_id
        WHERE o.client_id = c.id
        ORDER BY o.date_commande DESC
        LIMIT 1) AS dernier_produit
FROM clients c;
```

Signal à retenir : une sous-requête placée comme une valeur unique doit garantir qu'elle ne renvoie jamais plus d'une ligne. Ajoute ORDER BY et LIMIT 1, ou passe par une agrégation.

```
SELECT client_id FROM clients_actifs
UNION
SELECT client_id FROM clients_vip;
```

25. UNION vs UNION ALL PIÈGE CLASSIQUE

Pourquoi ça casse : UNION dédouble automatiquement les lignes identiques entre les deux listes. Un client à la fois actif et VIP n'apparaît qu'une fois : un COUNT(*) posé derrière sous-compte les statuts cumulés voulus.

```
SELECT client_id FROM clients_actifs
UNION ALL
SELECT client_id FROM clients_vip;
```

Signal à retenir : UNION dédouble et coûte un tri interne, UNION ALL garde tout sans comparer. N'utilise UNION que si le doublon est vraiment une erreur à corriger.

Dates, fenêtres et tri.

Ici, le piège se glisse dans une borne, un alias ou un ex-aequo, pas dans une erreur de syntaxe.

```
SELECT *
FROM commandes
WHERE date_commande BETWEEN '2026-06-01' AND '2026-06-30';
```

26. BETWEEN sur des dates PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : si **date_commande** est un TIMESTAMP, une commande passée le 30 juin à 14h est exclue : BETWEEN compare à minuit ('2026-06-30 00:00:00'). La borne haute écrête silencieusement la dernière journée.

```
SELECT *
FROM commandes
WHERE date_commande >= '2026-06-01'
      AND date_commande < '2026-07-01';
```

Signal à retenir : pour un intervalle de dates sur un TIMESTAMP, préfère « >= début et < début du jour suivant » plutôt que BETWEEN, qui piège sur l'heure de la borne finale.

```
SELECT client_id, SUM(montant) AS client_id
FROM commandes
GROUP BY client_id
ORDER BY client_id;
```

27. ORDER BY sur alias PIÈGE CLASSIQUE

Pourquoi ça casse : l'alias **client_id** porte le même nom que la vraie colonne **client_id** : PostgreSQL peut lever une ambiguïté, ou trier sur la mauvaise valeur, sans que rien ne l'indique dans le SELECT.

```
SELECT client_id, SUM(montant) AS total_client
FROM commandes
GROUP BY client_id
ORDER BY total_client DESC;
```

Signal à retenir : ne nomme jamais un alias comme une colonne existante de la requête. Portabilité : SQL Server et MySQL tolèrent aussi l'alias en ORDER BY, mais pas toujours en GROUP BY ou HAVING.

```
SELECT *
FROM commandes
LIMIT 5;
```

28. LIMIT sans ORDER BY PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : sans ORDER BY, PostgreSQL ne garantit aucun ordre stable. Les « 5 premières lignes » peuvent changer d'une exécution à l'autre selon le plan d'exécution, même sans modification des données.

```
SELECT *
FROM commandes
ORDER BY date_commande DESC
LIMIT 5;
```

Signal à retenir : LIMIT sans ORDER BY, c'est un résultat au hasard qui a l'air déterministe. Toujours accoler un ORDER BY explicite à un LIMIT.

```
SELECT id, client_id, date_commande,
       ROW_NUMBER() OVER (ORDER BY date_commande DESC) AS rn
FROM commandes;
-- puis WHERE rn = 1
```

29. ROW_NUMBER sans PARTITION BY correcte PIÈGE CLASSIQUE

Pourquoi ça casse : sans PARTITION BY client_id, la numérotation porte sur toute la table. rn = 1 ne renvoie que la commande la plus récente tous clients confondus, pas une par client.

```
SELECT id, client_id, date_commande, rn
FROM (
  SELECT id, client_id, date_commande,
         ROW_NUMBER() OVER (PARTITION BY client_id
                           ORDER BY date_commande DESC) AS rn
  FROM commandes
) t
WHERE rn = 1;
```

Signal à retenir : dès qu'on veut « le premier par groupe », PARTITION BY doit contenir la clé du groupe. Sans lui, la fenêtre porte sur tout le tableau.

```
SELECT nom, departement, salaire,  
       ROW_NUMBER() OVER (PARTITION BY departement  
                           ORDER BY salaire DESC) AS rang  
FROM employes;  
-- puis WHERE rang = 1
```

30. RANK vs ROW_NUMBER sur doublons PIÈGE ÉLIMINATOIRE

Pourquoi ça casse : si deux employés du même département ont exactement le même salaire, ROW_NUMBER attribue quand même des rangs 1 et 2 distincts, de façon arbitraire selon l'ordre physique des lignes. Un ex-aequo légitime disparaît du résultat « meilleur salaire ».

```
SELECT nom, departement, salaire, rang  
FROM (  
  SELECT nom, departement, salaire,  
         RANK() OVER (PARTITION BY departement  
                     ORDER BY salaire DESC) AS rang  
  FROM employes  
) t  
WHERE rang = 1;
```

Signal à retenir : pour un classement avec égalités à traiter équitablement, utilise RANK ou DENSE_RANK, pas ROW_NUMBER, qui force une unicité arbitraire.

À retenir avant le test

Les réflexes transversaux des 30 pièges, en une checklist.

NULL

Jamais = NULL, jamais NOT IN sur une colonne nullable.

Jointure

Vérifie le grain avant de sommer ou compter.

Agrégat

HAVING pour un agrégat, jamais WHERE.

Tri

Jamais de LIMIT sans ORDER BY.

Fenêtre

PARTITION BY doit contenir la clé du groupe.

Doute

Une requête qui tourne sans erreur n'est pas forcément juste.

30 pièges corrigés. Tu n'as pas besoin de mémoriser chaque requête au mot près. Retiens surtout le geste : douter d'un chiffre qui semble juste, vérifier le grain d'une jointure, tester les NULL avant tout agrégat. C'est ce réflexe, plus que la syntaxe, que le test technique cherche à voir.

Fait main, en solo.

Si ce guide t'a aidé à sécuriser un test technique, partage-le autour de toi : il peut débloquent un autre candidat. Et si tu veux que je traite un piège précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

